

LEVEL 4 — DEVELOPMENT TRACK FOUNDATIONS

Testing

Automated checks that code and data behave as expected — and keep behaving as the codebase changes

Covers: Unit tests · integration tests · regression tests · property-based testing · backtest validation · data validation

Table of Contents

Right-click and choose “Update Field” (or “Update entire table”) after opening in Word to populate page numbers.

Table of Contents	2
11. Testing (Level 4)	3
11.1 Learning Position at a Glance	3
11.2 Unit Tests	3
11.3 Integration Tests	4
11.4 Regression Tests	4
11.5 Property-Based Testing	5
11.6 Backtest Validation	6
11.7 Data Validation	7
11.8 Worked Exercise — Model Answer	8
11.9 Common Mistakes — Deeper Look	9
11.10 Advanced Extensions	9
11.11 Professional Applications	10

11. Testing (Level 4)

Covers: Unit tests · integration tests · regression tests · property-based testing · backtest validation · data validation

11.1 Learning Position at a Glance

- **What it is:** Automated checks that code and data behave as expected, and continue to do so as the codebase changes.
- **Why it matters:** Silent errors in return calculation, signal construction or risk metrics are extremely costly and easy to introduce without automated tests catching them.
- **Prerequisites:** Software Engineering Foundations.
- **Connects to Research:** Regression tests on a backtesting engine catch accidental changes to strategy logic when refactoring.
- **Connects to Development:** This is a core development competency; untested trading code should not reach production.
- **Connects to Trading:** A tested execution or risk-limit system is what prevents a code bug from becoming a real loss.
- **Practical exercise:** Write unit tests for a returns-calculation module covering edge cases (zero prices, missing data, single-observation series).
- **Common beginner mistakes:**
 - Testing only the ‘happy path’ and ignoring edge cases like missing or malformed data.
 - Treating a backtest's historical Sharpe ratio as a substitute for actually testing the code's correctness.
- **Advanced extensions:** Property-based testing (e.g. Hypothesis) for numerical code; continuous integration pipelines.
- **Professional applications:** Backtesting-engine reliability; production trading-system safety.

The testing pyramid: more, faster, cheaper tests at the base

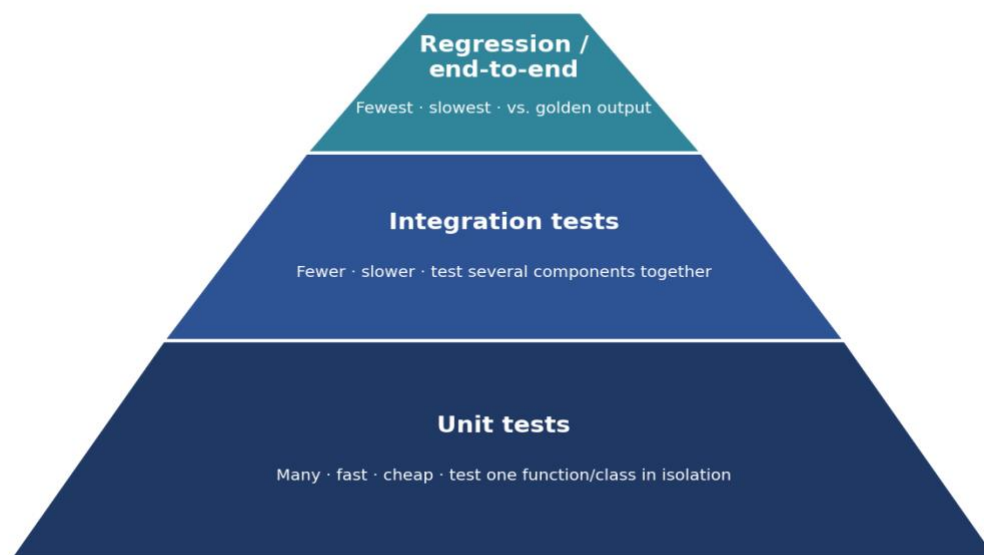


Figure 11.1 — The testing pyramid: a healthy test suite has many fast, cheap unit tests at the base, fewer integration tests in the middle, and only a small number of slow, comprehensive regression/end-to-end tests at the top.

11.2 Unit Tests

A unit test checks one small piece of behaviour — typically a single function or method — in isolation from the rest of the system. Unit tests are fast (milliseconds each), numerous (a healthy codebase might have thousands), and specific: when one fails, it points directly at the function responsible, rather than leaving the diagnosis to a slow manual investigation.

Example 11.1 — Two focused unit tests for the `sharpe_ratio` function from Example 9.9

```
import pytest
from metrics import sharpe_ratio
import pandas as pd

def test_sharpe_ratio_positive_returns():
    returns = pd.Series([0.01, 0.02, -0.005, 0.015])
    result = sharpe_ratio(returns, periods_per_year=252)
    assert result > 0

def test_sharpe_ratio_zero_variance_returns_nan():
    returns = pd.Series([0.01, 0.01, 0.01, 0.01])
    assert pd.isna(sharpe_ratio(returns))
```

- **Fast:** A full unit test suite should run in seconds, not minutes, so it can be run constantly during development without breaking flow.
- **Isolated:** A unit test should not depend on a live database, network connection, or another test's side effects; anything external is replaced with a controlled stand-in (a mock or fixture).
- **Specific:** One test checks one behaviour; a test with many unrelated assertions makes it unclear, on failure, which specific behaviour actually broke.

11.3 Integration Tests

An integration test checks that several components work correctly together — exactly the kind of interaction that unit tests, by design, do not cover because each unit test isolates a single piece. For the Signal/Portfolio/Backtester design from §9.10, a unit test might check that `MomentumSignal.generate()` produces correct weights in isolation, while an integration test checks that a Backtester wired up with a real Signal and a real Portfolio produces sensible end-to-end output.

Example 11.2 — An integration test exercising Signal, Portfolio and Backtester together

```
def test_backtester_runs_end_to_end_without_error():
    prices = load_sample_prices()          # a small, fixed test dataset
    signal = MomentumSignal(lookback=20)
    portfolio = Portfolio(cash=1_000_000.0)
    backtester = Backtester(signal=signal, portfolio=portfolio)

    result = backtester.run(prices)

    assert result.cash >= 0
    assert set(result.positions.keys()).issubset(set(prices.columns))
```

Integration tests are inherently slower and fewer in number than unit tests (per Figure 11.1) because they exercise more code and often more realistic, larger inputs; a healthy suite uses them to catch problems specifically at the seams between components, while relying on the much larger unit-test base to cover each component's internal logic thoroughly.

11.4 Regression Tests

A regression test protects against a specific class of bug: a change that unintentionally alters behaviour that was previously correct. The standard technique is to save a trusted, known-good output (a 'golden' file) generated by a

version of the code already validated to be correct, and to fail automatically if a later change produces a different result without that difference being deliberately reviewed and accepted.

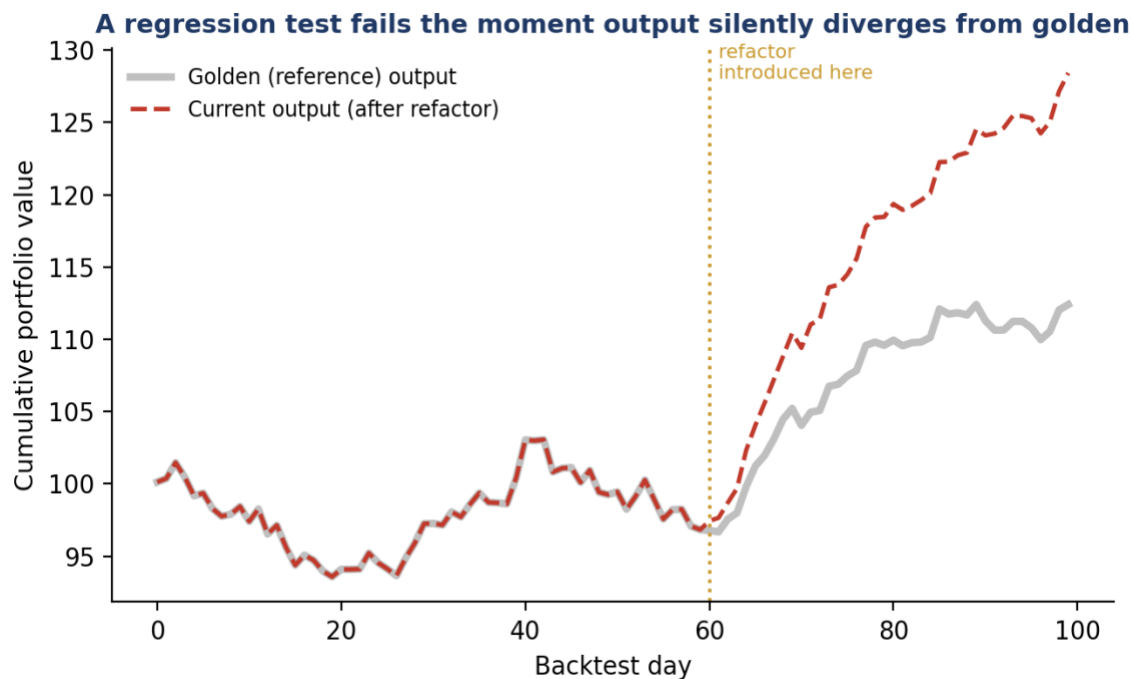


Figure 11.3 — A regression test compares current output against a golden reference on every change; the moment a refactor silently alters behaviour, the comparison fails immediately, rather than the change reaching production undetected.

Example 11.3 — A regression test against a saved golden backtest output

```
def test_backtest_matches_golden_output():
    prices = load_sample_prices()
    result = run_backtest(MomentumSignal(20), prices)

    golden = pd.read_csv("tests/golden/momentum_backtest_v1.csv")
    pd.testing.assert_frame_equal(
        result, golden, check_exact=False, rtol=1e-6,
        obj="Backtest output no longer matches the golden reference "
        "-- if this change is intentional, regenerate the golden "
        "file and describe why in the pull request.",
    )
```

This is precisely the safeguard that lets a backtesting engine be refactored (for clarity, performance, or to add a feature) with confidence: any accidental change to strategy logic — not just an outright crash — is caught immediately, directly supporting the reproducibility discipline from §6.5 and §10.6.

11.5 Property-Based Testing

A conventional (example-based) test checks specific, hand-picked inputs the author thought to try. Property-based testing instead states a general property that should hold for all valid inputs — for example, 'a computed weight vector should always sum to at most 1.0' — and lets a tool (such as the Hypothesis library) automatically generate hundreds of varied, including deliberately extreme and adversarial, inputs to try to find a case where the property fails.

Property-based testing explores far more of the input space

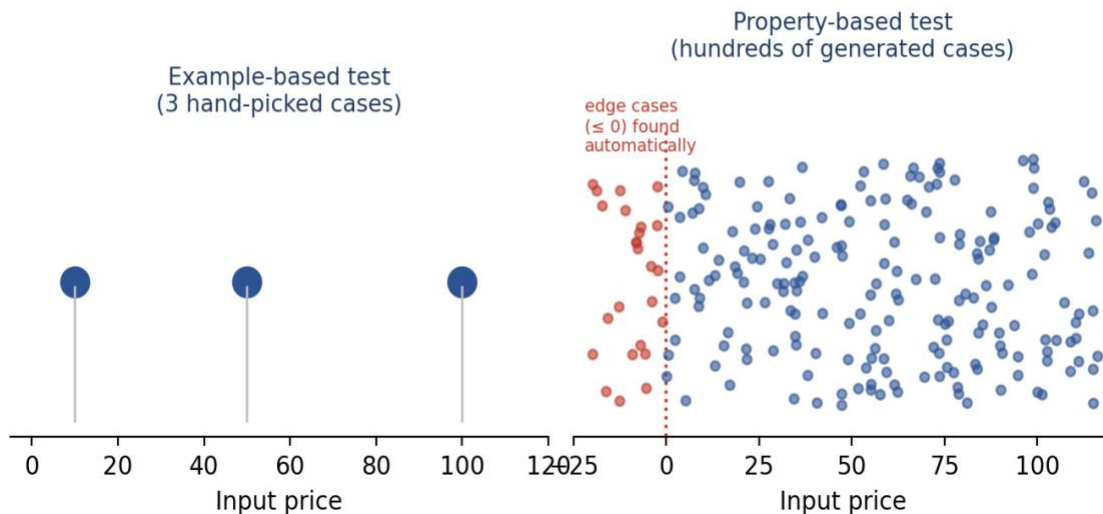


Figure 11.2 — A hand-written example-based test tries a handful of cases the author thought of; a property-based test explores a far wider swath of the input space automatically, including edge cases (here, non-positive prices) the author may never have thought to write down.

Example 11.4 — Property-based tests for the `position_size` function from Example 9.5

```
from hypothesis import given, strategies as st

@given(st.floats(min_value=-1000, max_value=1000))
def test_position_size_never_negative_for_positive_price(price):
    if price <= 0:
        return # invalid input is expected to raise, tested separately
    size = position_size(capital=100_000, price=price, risk_pct=0.02)
    assert size >= 0

@given(st.floats(max_value=0))
def test_position_size_rejects_non_positive_price(price):
    with pytest.raises(ValueError):
        position_size(capital=100_000, price=price, risk_pct=0.02)
```

Property-based testing is particularly well suited to numerical quant code precisely because the input space (prices, returns, weights) is large and continuous, and the properties that matter — weights summing correctly, no negative sizes, no division by zero — are naturally expressed as general rules rather than a finite list of examples.

11.6 Backtest Validation

Backtest validation is testing applied specifically to a backtesting engine's own correctness — a different concern from whether a particular strategy is profitable (covered in §6.6–§6.7). A backtesting engine can be completely wrong in a way that a Sharpe ratio alone will never reveal: a look-ahead bug that leaks future information can produce an excellent, entirely fictitious backtest Sharpe ratio.

- **Known-answer tests:** Construct a small, synthetic dataset where the correct output can be computed by hand (e.g. a single asset with a perfectly predictable price path), and assert the engine reproduces that known answer exactly.

- **Look-ahead detection:** A dedicated test that shifts a feature's timestamp forward by one period and confirms the reported backtest performance changes — if performance is suspiciously unaffected by a feature's information no longer being available in time, that is itself a warning sign of a pre-existing look-ahead bug.
- **Cost sensitivity checks:** Re-run the same backtest with transaction costs doubled and with them set to zero; a real, robust edge should survive reasonable cost assumptions, and an implausibly large sensitivity to a cost assumption warrants scrutiny before trusting the result.

Example 11.5 — A deliberate look-ahead-bias check on the backtesting engine itself

```
def test_backtest_detects_lookahead_if_introduced():
    prices = load_sample_prices()
    honest_result = run_backtest(MomentumSignal(20), prices)

    # Deliberately shift the signal to use tomorrow's price -- a bug we
    # are testing our detection process would catch, not a real feature.
    leaky_prices = prices.shift(-1)
    leaky_result = run_backtest(MomentumSignal(20), leaky_prices)

    # A leaky (look-ahead) backtest should look suspiciously better;
    # if it doesn't, the engine may already have a discipline that
    # tolerates leakage, which itself deserves investigation.
    assert leaky_result.sharpe_ratio > honest_result.sharpe_ratio
```

11.7 Data Validation

Data validation checks that incoming data itself is well-formed and plausible before it is used, rather than trusting that a data feed always behaves as documented. This is the practical, automated enforcement of the data-quality discipline introduced in §6.4: a schema check catches a missing or renamed column, a range check catches an impossible value (a negative price, a 500% single-day return), and a freshness check catches data that has silently stopped updating.

A data validation pipeline gates bad data before it reaches research

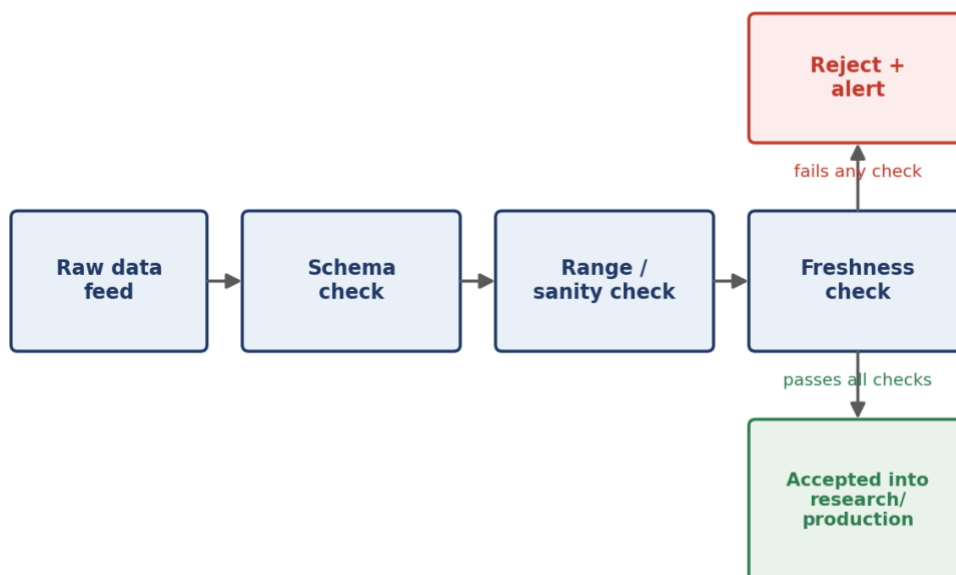


Figure 11.4 — Data validation as a pipeline: each stage can reject and alert independently, so bad data is caught at the earliest possible point rather than silently propagating into research or a live trading decision.

Example 11.6 — Schema, range and freshness checks in a single validation function

```
def validate_price_data(df: pd.DataFrame) -> None:
```

```

required_columns = {"date", "ticker", "close"}
if not required_columns.issubset(df.columns):
    raise DataValidationError(f"Missing columns: {required_columns - set(df.columns)}")

if (df["close"] <= 0).any():
    raise DataValidationError("Non-positive price found")

daily_return = df.groupby("ticker")["close"].pct_change()
if (daily_return.abs() > 0.5).any():
    logger.warning("Return exceeding 50%% detected -- possible bad tick")

latest_date = df["date"].max()
if (pd.Timestamp.now() - latest_date).days > 3:
    raise DataValidationError(f"Data is stale: latest date is {latest_date}")

```

11.8 Worked Exercise — Model Answer

Exercise: Write unit tests for a returns-calculation module covering edge cases (zero prices, missing data, single-observation series).

The function under test

```

def simple_returns(prices: pd.Series) -> pd.Series:
    """Compute simple period-over-period returns from a price series."""
    if len(prices) < 2:
        raise ValueError("Need at least two observations to compute a return")
    if (prices <= 0).any():
        raise ValueError("Prices must be strictly positive")
    return prices.pct_change().dropna()

```

Model answer — covering the happy path and every stated edge case explicitly

```

import pytest
import pandas as pd
from returns import simple_returns

def test_simple_returns_happy_path():
    prices = pd.Series([100, 110, 99])
    result = simple_returns(prices)
    assert result.iloc[0] == pytest.approx(0.10)
    assert result.iloc[1] == pytest.approx(-0.10, rel=0.01)

def test_simple_returns_rejects_zero_price():
    prices = pd.Series([100, 0, 99])
    with pytest.raises(ValueError, match="strictly positive"):
        simple_returns(prices)

def test_simple_returns_rejects_negative_price():
    prices = pd.Series([100, -5, 99])
    with pytest.raises(ValueError, match="strictly positive"):
        simple_returns(prices)

def test_simple_returns_handles_missing_data():
    prices = pd.Series([100, None, 99])
    # NaN propagates rather than silently being treated as zero:
    result = simple_returns(prices)
    assert result.isna().any()

def test_simple_returns_rejects_single_observation():
    prices = pd.Series([100])

```



```

with pytest.raises(ValueError, match="at least two observations"):
    simple_returns(prices)

def test_simple_returns_rejects_empty_series():
    prices = pd.Series([], dtype=float)
    with pytest.raises(ValueError, match="at least two observations"):
        simple_returns(prices)

```

Notice the shape of this test suite: one test confirms ordinary, expected behaviour, and five more each target a single, specific edge case named in the exercise (zero price, negative price, missing data, single observation) plus one the exercise implies but does not name explicitly (an empty series) — exactly the habit of actively hunting for untested edge cases that §11.9 discusses next.

11.9 Common Mistakes — Deeper Look

Mistake: Testing only the ‘happy path’ and ignoring edge cases like missing or malformed data

A test suite that only exercises well-formed, typical inputs provides real but narrow reassurance: it confirms the code works when everything goes right, which is exactly the situation least likely to cause a production incident. The failures that matter in practice — a data feed returning null, an empty DataFrame after an overly aggressive filter, a single-row series at the start of a new instrument's history — are precisely the cases a happy-path-only suite never touches. The worked exercise in §11.8 is structured the way it is specifically to counter this habit: for every function, the disciplined question is not just 'does this work for typical input?' but 'what is the shortest list of unusual, boundary or invalid inputs I have not yet tried?'

Mistake: Treating a backtest's historical Sharpe ratio as a substitute for actually testing the code's correctness

A high Sharpe ratio and correct code are entirely different claims, and one provides no evidence at all for the other: a backtesting engine with a look-ahead bug (§11.6) can produce an excellent Sharpe ratio precisely because it is wrong, and a bug-free engine testing a genuinely weak strategy will correctly report a poor one. Confusing the two leads to a dangerous inference — 'the numbers look great, so the code must be fine' — that runs in exactly the wrong direction; correctness must be established independently, through the kinds of tests in §11.2–§11.6, before a backtest's performance numbers can be trusted at all.

⚠ Case study: an excellent Sharpe ratio that was actually a bug

- A researcher reported an unusually strong backtest Sharpe ratio for a new signal and, given the strength of the result, moved quickly toward sizing it for live deployment without further scrutiny of the underlying code.
- A colleague reviewing the pull request ran the look-ahead detection check from Example 11.5 as a matter of routine and found that shifting the signal's input data by one day barely changed the reported performance — exactly the suspicious insensitivity to timing that indicates leaked future information.
- The signal was in fact silently using a data field that was only available with a one-day lag but had been joined onto the backtest as if available same-day; once corrected, the strategy's true edge was materially smaller — the high Sharpe ratio had been evidence of a bug, not of skill.

11.10 Advanced Extensions

- **Property-based testing (e.g. Hypothesis) for numerical code:** Beyond the introduction in §11.5, mature use includes Hypothesis's ability to automatically shrink a failing example to the smallest, simplest input that still reproduces the failure, and 'stateful' testing that generates entire sequences of operations (not just single inputs) to test components like Portfolio that accumulate state over many calls.
- **Continuous integration pipelines:** Extending the CI configuration introduced in §10.7 to run the full testing pyramid (Figure 11.1) automatically on every pull request — fast unit tests on every commit, slower integration and regression tests before merge — so that the cost-of-change curve from Figure 9.1 is kept as flat as possible: every defect is caught by an automated test before a human reviewer, and every defect a human reviewer catches is caught before production.

11.11 Professional Applications

- **Backtesting-engine reliability:** The full suite from §11.2–§11.6 — unit tests on individual calculations, integration tests across Signal/Portfolio/Backtester, regression tests against golden output, and dedicated look-ahead and cost-sensitivity checks — is what allows a firm to trust that a reported backtest result reflects a real strategy property rather than an artefact of engine bugs.
- **Production trading-system safety:** Data validation (§11.7) and comprehensive test coverage of risk and execution logic are the concrete, automated version of the standing warning that opened this section: that a single silent bug in position-sizing or risk logic can lose real money. Testing is what turns that risk from a matter of hope into a matter of verified, continuously-checked fact.